

Verifier-Grounded Generate-Verify-Repair for Intent→Configuration NetOps Tasks: A Paired Mininet Emulation Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a provenance-traced paired confirmatory study (N = 200 intents; 400 paired episodes) to evaluate whether a deterministic verifier-grounded generate-verify-repair (GVR) loop reduces operationally detectable policy-violation errors when a hosted LLM produces device configuration sequences from operator intent in a Mininet emulation. Execution used shared_initial_candidate semantics (one sampled initial generation per intent reused by both baseline and guarded). Baseline success (no detected policy violation) = 196/200 (98%); guarded success = 200/200 (100%). Paired risk difference (guarded - baseline) = 0.02; preregistered exact McNemar two-sided p = 0.125 (primary confirmatory test). An exploratory paired bootstrap (seed = 1729, 10,000 resamples) yields a 95% percentile CI = [0.005, 0.04]. Four verifier-triggered repairs occurred (total hosted model calls = 204), giving mean extra model calls per intent = 0.02 and mean extra calls per avoided violation = 1.0. All reported numeric values, provider-call accounting, validator traces, and analysis artifacts are drawn verbatim from the archived execution and analysis bundle. We interpret the preregistered confirmatory result conservatively and discuss construct, internal, and external validity limits and operational implications.

I. INTRODUCTION

Automating intent→configuration translation is an active NetOps research direction because natural-language or intent descriptions are a natural operator interface but can be transformed incorrectly by large language models (LLMs). Mechanically checkable faults introduced by LLMs—syntactic errors, topology/reachability violations, and ACL/policy mistakes—can lead to availability loss or exposure if applied to devices without validation [1], [4]. Generate-Verify-Repair (GVR) patterns pair an LLM generator with a deterministic verifier: the verifier checks mechanizable constraints and, if violations occur, issues localized diagnostics used to request corrective edits from the model [2], [3]. Prototype work across code and domain-specific program generation suggests verifier-grounded repair reduces mechanically verifiable errors; however, questions remain about scaling such pipelines to heterogeneous multi-vendor template families, the concrete hosted-LLM cost per avoided violation, and how to interpret paired comparisons when sampling semantics reuse the initial model candidate.

Research question and contribution. Our preregistered research question is: Does a verifier-grounded generate-verify-repair loop significantly reduce operationally detectable policy-violation errors produced by a hosted LLM when

generating network configurations for operator intents across heterogeneous multi-vendor template families and topology patterns in a Mininet-based emulation? Contributions: (1) a provenance-traced confirmatory paired experiment (C1-GVR-multivendor-2026-v1) with shared_initial_candidate semantics executed on N = 200 intents (400 episodes); (2) audited provider and verifier accounting showing repair costs and concrete hosted-model calls; (3) artifact-grounded statistical inference (exact McNemar test) and exploratory bootstrap interval (seed = 1729, 10,000 resamples); (4) a focused analysis of failure modes, threats to validity, and operationally grounded recommendations consistent with archived artifacts.

II. RELATED WORK

Recent literature on LLMs for NetOps and AIOps converges on two observations that are directly relevant to verifier-grounded pipelines. First, surveys and domain reviews document rapid uptake of LLMs for network tasks while highlighting recurring gaps in evaluation practice: missing provenance, limited verifier integration, and scarce multi-vendor or emulation-scale evidence [5], [6]. These reviews synthesize heterogeneous reports (research prototypes, bench demonstrations, and applied surveys) and conclude that while LLMs reduce human effort for drafting configurations, the community lacks standardized, provenance-traced benchmarks that pair generators with deterministically auditable verifiers. This limits cross-study comparability and weakens claims about operational safety or generalization across vendors and topologies [5], [6].

Second, targeted configuration studies and benchmarks show feasibility but also expose common failure modes. Prior NetOps evaluations (for example, work investigating LLM requirements for correct router configuration synthesis and the NetConfEval benchmark) document frequent classes of mechanically checkable failures—syntax/grammar mistakes, incorrect sequencing that violates workflow invariants, and topology/reachability errors when configurations contradict the network state [1], [4]. These studies typically evaluate generator accuracy on curated tasks and emphasize the need for automated checking; however, many prototype benchmarks either omit a closed-loop repair stage or do not provide provenance traces linking initial outputs, verifier diagnostics, and any applied repairs. In practice, reported improvements in such studies are therefore difficult to attribute to deterministic

validator interventions versus differences in prompting or sampling.

A cross-domain body of work motivates verifier-grounded mitigation architectures (generate–verify–repair). Recent methodological proposals and prototype demonstrations in code and domain-specific program generation argue that a verifier capable of expressing mechanizable constraints plus localized diagnostic feedback enables targeted repairs that reduce mechanically detectable errors [2], [3]. These works provide conceptual and single-case empirical support: they show that when a verifier can (i) deterministically check the artifact against explicit constraints and (ii) return actionable, localized diagnostics, a bounded repair policy can converge failing samples to valid artifacts. Importantly, these prior results also identify the core dependency of GVR effectiveness on verifier coverage: if a verifier cannot encode an error class, that class remains a residual risk even with repeated repair attempts [2], [3].

How our study relates. Building on the empirical observations and methodological proposals above, our experiment addresses a concrete gap: scalable, provenance-traced paired evaluations that measure the marginal effect of deterministic verification and bounded repair across heterogeneous template families and topology patterns. Unlike prior NetOps benchmarks that either evaluate single-pass generation or lack explicit provenance between generator outputs and verifier actions [1], [4], or like GVR proposals that focus on single-domain demonstrations [2], [3], our paired design (with `shared_initial_candidate` semantics) isolates the verifier/repair contribution by reusing the identical initial LLM sample for both conditions and then measuring the final guarded outcome after permissive, verifier-triggered repair. This combination—multi-vendor synthetic templates exercised in Mininet, deterministic verifier traces, provable provider-call accounting, and preregistered paired inference—directly responds to the literature’s call for provenance and auditable assessments of verifier-assisted generation [5], [6]. At the same time, we adopt the caution emphasized in the surveyed and prototype literature: verifier expressivity bounds the classes of errors that can be detected and repaired, and mechanical verification cannot by itself guarantee absence of higher-order semantic misinterpretations outside the verifier’s rule set [2], [3], [5].

III. METHODOLOGY

Preregistration and formal estimand. We preregistered the study as C1-GVR-multivendor-2026-v1 before any model calls (preregistration SHA-256: 9e0fc992665835975b01289f7841926ef6abf3584252c5d0459ba0ecf166bb50). The primary estimand is the `paired_success_rate_difference_guarded_minus_baseline`: the per-intent difference in the probability that an intent produces zero operationally detectable policy violations under guarded (GVR) relative to baseline (LLM-only). The confirmatory test declared in the preregistration is the exact McNemar two-sided test at $\alpha = 0.05$ applied to the paired binary success indicators.

Sampling, stratification, and fixed sampling semantics. Task selection was deterministic and stratified across three synthetic vendor-template families and four topology patterns (12 strata). $N = 200$ unique operator intents were fixed by the deterministic task generator before any model calls (`task_index_profile` selection is recorded in the execution manifest). The preregistration and adapter execution contract explicitly used `shared_initial_candidate` semantics: exactly one sampled initial generation per intent was produced and this identical initial candidate was reused by both baseline and guarded episodes. Under these semantics the baseline outcome equals the deterministic validator verdict applied to that shared initial candidate; the guarded outcome equals the final result of the guarded pipeline after deterministic validation and any permitted repair. The preregistration also declared that independent constrained-prompt arms were not executed; therefore any within-pair differences can arise only from deterministic validator actions and permitted repair calls, not from different first-pass samples or different initial prompts.

Model configuration, sampling notation, and budgetary constraints. The declared archived model is `gpt-5-mini` (provider recorded as `openai_responses`) and the adapter enforced a strict call budget: `planned_episode_count = 400`, `initial_generation_calls_per_task = 1`, `maximum_repair_calls_per_task = 1`, `maximum_model_calls = 400`. The archived execution `model_configuration.json` records `temperature = null` (unset). We explicitly state: `temperature = null/unset` is not evidence of deterministic hosted-model sampling and does not imply `temperature = 0`. Because provider sampling semantics are external to the adapter, we treat the single shared initial generation per intent as the executed stochastic sample; subsequent guarded repairs (if any) are additional provider calls permitted only by the predeclared repair policy.

Deterministic verifier design and scoring rule. The `deterministic_netops_validator_v5` is a rule-based verifier implemented to mechanistically evaluate intent→configuration candidates. Its functional components (all coded as deterministic checks applied to the candidate configuration and the emulated Mininet state) include: 1) syntactic parsing and command-pattern conformance (token-level grammar and per-vendor command templates); 2) required-command sequencing and workflow policy checks driven by the per-task payload (e.g., ordered steps, protected interfaces, minimum-active constraints); 3) reachability and topology consistency checks that compare the candidate’s intended final state against the emulated network topology and device states; 4) ACL and policy encodings derived from task payload constraints (checks for forbidden field changes or prohibited sequences that would violate policy). The verifier returns a structured trace: `validation_before` (diagnostic list and violation codes), a binary validity verdict (`valid: true/false`), and, when violations are present and repair is permitted by policy, a localized diagnostic summary that is included verbatim in the repair prompt. The scoring rule `full_intent_constraint_validation` yields `success = 1` only when

the final validated configuration satisfies all mechanizable constraints.

Repair policy and prompt semantics. The guarded pipeline applies a conservative bounded repair policy as preregistered: if and only if the verifier reports violations on the shared initial candidate, at most one verifier-triggered repair call may be issued. Repair prompts include (a) the original shared initial candidate, (b) the verifier’s localized diagnostic messages (violation codes and minimal context identifying implicated commands/fields), and (c) an explicit instruction to produce a corrected configuration that satisfies the listed constraints. Repairs were allowed only when the verifier flagged violations and were restricted to a single additional hosted-model call per intent to preserve the predeclared budget and to evaluate cost per avoided violation.

Scoring, exclusions, and missingness treatment. The primary confirmatory analysis treats the pairwise binary success indicators (no detected violations after the pipeline completes for baseline and guarded) per the preregistration. Predeclared exclusion rules removed duplicate tasks detected via deterministic prompt+context hashing and episode pairs aborted due to unrecoverable infrastructure/container substitution failures; such exclusions would be reported (none occurred in the archived run). Ambiguous verifier outputs (diagnostics flagged by deterministic ambiguity rules) were conservatively treated as violations in the primary analysis (this conservative rule was preregistered). The missingness plan treated genuine infrastructure/API failures as missing-at-random for the primary test; a sensitivity analysis treating failures as conservative failures was predeclared.

Analysis plan and resampling specifics. The preregistered primary inferential procedure is the exact McNemar two-sided test applied to the 2×2 paired contingency table (n_{11} , n_{10} , n_{01} , n_{00}). Secondary and exploratory estimators declared in the preregistration include: (i) a paired bootstrap percentile 95% confidence interval for the paired risk difference (bootstrap_seed = 1729; bootstrap_resamples = 10,000), and (ii) descriptive cost metrics (mean extra hosted model calls per intent and mean extra calls per avoided violation). We note that bootstrap percentile intervals can be unstable when discordant pair counts ($n_{10} + n_{01}$) are small; this instability motivated treating the McNemar exact test as the formal confirmatory inference and labeling the bootstrap interval exploratory in the manuscript.

Provider accounting, auditability, and reproducibility artifacts. The experiment’s adapter recorded a provider-call audit: planned_maximum_model_calls = 400; hosted_model_call_event_count was auditable in the execution manifest (the adapter records stage_counts and condition_counts). The preregistration required recording all raw provider responses, verifier traces, repair prompts, deterministic task manifests, and an execution manifest; these artifacts were archived for machine verification. Analysis code, seeds, and exact resampling parameters were archived to enable deterministic replication of the reported numerical analyses.

Why shared_initial_candidate semantics matters for causal interpretation. We emphasize that because the archived run used shared_initial_candidate semantics and permitted at most one repair call per intent, the paired comparison isolates the effect of deterministic verification and the bounded repair step on the shared initial sample. In this design baseline is not an independent generation arm; it is the validator’s direct verdict on the sampled candidate. Any between-condition improvements within a pair therefore must arise from validator-triggered repair converting a failing shared initial candidate into a passing final configuration, not from differing initial samples or alternative prompting.

Implementation and operational controls. The adapter enforced deterministic task ordering and recorded task indices prior to model calls. Repair prompts were templated and deterministic given the verifier diagnostics; repair calls were logged with the exact repair prompt text and provider response. The deterministic verifier and task generator versions used in the run (deterministic_netops_validator_v5 and deterministic_netops_task_generator_v5) are recorded in the archived model and artifact manifests to permit exact replay of the pipeline on the same synthetic task bank and emulation state. All numerical quantities reported in Results (model_call counts, stage_counts, and contingency table cells) are drawn from those archived manifests and reconciled in the deterministic_reconciliation artifact.

IV. RESULTS

Execution and audited provider accounting. The preregistered run completed as planned: planned_episode_count = 400; completed_episode_count = 400; complete_pair_count = 200; failed_episode_count = 0. Deterministic provider accounting (execution manifest) reports hosted_model_call_event_count = 204 decomposed into 200 shared_initial_generation events and 4 repair calls; cache_hit_count = 0; cache_miss_count = 204; stage_counts = {shared_initial_generation: 200, repair: 4}. These audit counts are reported verbatim from the archived execution manifest.

Paired outcomes and confirmatory test. The paired 2×2 contingency (analysis/paired_contingency_table.csv) is: $n_{11} = 196$ (both succeed), $n_{10} = 4$ (guarded succeed, baseline fail), $n_{01} = 0$ (guarded fail, baseline succeed), $n_{00} = 0$ (both fail). Baseline success rate = $196/200 = 0.98$; guarded success rate = $200/200 = 1.0$; paired risk difference (guarded - baseline) = 0.02. The preregistered exact McNemar two-sided test yields $p = 0.125$ (primary confirmatory result; do not reject at $\alpha = 0.05$).

Exploratory interval and bootstrap caution. The archived exploratory paired bootstrap (bootstrap_seed = 1729; resamples = 10,000) produced a 95% percentile CI for the paired risk difference = [0.005, 0.04]. We report this interval as exploratory and caution that bootstrap percentile intervals can be unstable when discordant pair counts ($n_{10} + n_{01}$) are small; the exact McNemar test remains the formal preregistered inference. The bootstrap seed and resampling count are archived (analysis_log.jsonl).

Repair events and cost accounting. Four verifier-triggered repairs were invoked across 200 intents. Total hosted model calls = 204 (200 shared initial + 4 repairs), giving mean hosted model calls per intent = 1.02 and mean extra calls per intent due to repairs = 0.02. Each repair corrected one baseline failure in the archived contingency; thus mean extra calls per avoided violation = 1.0. These arithmetic derivations follow directly from the audited provider_call counts in deterministic_reconciliation and the execution manifest.

Representative artifacts and substantiation. Representative scoring artifacts (execution/scoring/task-000004-*, task-000005-*, task-000006-*) show identical shared_initial_candidate content and identical shared_initial responses reused by both baseline and guarded conditions. For the four discordant pairs, validator_trace.validation_before recorded detected violations and validation_after recorded that a repair was applied and the final configuration passed full_intent_constraint_validation. The raw shared_initial response files, validator traces, and repair decisions are archived and cited in the execution manifest; they substantiate the contingency counts and repair accounting above.

Power and interpretation. The preregistered power analysis targeted a 10 percentage point absolute reduction with $N \approx 157$ (McNemar approximation); $N = 200$ was chosen to permit stratified descriptions. The observed baseline failure prevalence (2%) was lower than some planning scenarios, reducing power to detect small absolute improvements and producing the small discordant count (4) that underlies the non-significant McNemar result. We therefore interpret the $p = 0.125$ as the definitive confirmatory outcome under the preregistered design and treat the bootstrap CI as supplementary magnitude information.

V. DISCUSSION

Causal interpretation under shared_initial_candidate semantics. Because execution used shared_initial_candidate semantics (one sampled initial generation per intent reused by baseline and guarded), paired improvements arise solely from deterministic validator/repair actions. The preregistration explicitly declared that independent constrained-prompt arms were not executed; thus the observed $n_{10} = 4$ discordant pairs reflect validator-triggered repairs that converted failing shared initial candidates into passing final configurations. Any statements attributing improvements to prompt-engineering or differing first-pass samples would be incorrect for this run.

Construct validity and verifier coverage. The deterministic_netops_validator_v5 covers syntactic correctness, required command sequencing, reachability/topology checks against the emulated state, and encoded ACL/policy checks derived from per-task payload constraints. This verifier can only detect violations expressible in its rules; higher-order semantic misinterpretations not captured by those encodings remain undetectable and are identified in our limitations. Increasing verifier expressivity (formal policy ontologies, broader semantic checks) is a direct next step to reduce residual false negatives.

Internal validity and nondeterminism. The archived model_configuration.json records temperature = null/unset; we reiterate that null/unset does not imply temperature = 0 or deterministic provider sampling. The execution manifest’s provider audit (hosted_model_call_event_count = 204, stage_counts, and condition_counts) was reconciled deterministically (all deterministic consistency checks passed) and confirms that cross-condition cache reuse did not occur; this audit supports the claim that baseline outcomes are the validator verdict on the identical shared initial generation used by guarded episodes.

Operational implications (bounded). Within the constrained Mininet emulation, synthetic templates, and validator rule set, a small repair budget (four repairs) removed the mechanically detectable baseline violations at very low hosted-model cost. This artifact-grounded observation suggests that when a deterministic verifier exists and provides localized actionable diagnostics, a bounded repair policy can remove template-driven mechanically checkable faults in similar emulated settings. We do not claim production readiness, multi-model generality, nor coverage for semantic errors outside the verifier.

Threats to validity and recommendations. Internal validity—shared_initial_candidate semantics were predeclared and executed; this prevents attributing gains to independent re-sampling. Construct validity—the verifier cannot detect all semantic errors; extend verifier coverage and measure false negatives. External validity—single declared model (gpt-5-mini) and synthetic templates in Mininet limit generalization; evaluate additional hosted models and hardware testbeds. Conclusion validity—small discordant counts reduce power; future preregistered work could target higher baseline failure prevalence scenarios or increase N . Based on archived evidence we recommend: (1) expanding deterministic verifier expressivity to formalize more semantic/policy ontologies; (2) performing paired designs that also execute independent multi-sample initial generations to separate prompt/sampling effects from repair effects; and (3) repeating provenance-traced evaluations across multiple hosted models and template families.

VI. LIMITATIONS

- Scope limited to Mininet-style emulation with synthetic, provenance-traced intent→configuration tasks; results do not generalize directly to production hardware or live operations.
- Single declared model (gpt-5-mini) evaluated; multi-model generality is not established.
- Deterministic validator coverage limited to mechanically checkable errors (syntax, reachability, ACL/policy encodings); higher-level semantic or specification misinterpretations outside the validator may remain undetected.
- Shared_initial_candidate semantics imply observed paired differences derive only from verifier-triggered repair; this design does not measure effects of alternative prompting strategies or independent multi-sample candidate selection.

- Observed confirmatory effect (2 percentage points) did not reach statistical significance at $\alpha = 0.05$ for $N = 200$ (exact McNemar $p = 0.125$); low baseline failure prevalence (2%) reduced power to detect small absolute improvements under some preregistered scenarios.

VII. CONCLUSION

We preregistered and executed a provenance-traced paired confirmatory study (C1-GVR-multivendor-2026-v1) using `shared_initial_candidate` semantics to measure whether a deterministic verifier plus bounded repair reduces mechanically detectable policy violations for intent→configuration tasks in a Mininet emulation. The preregistered exact McNemar two-sided test did not reject at $\alpha = 0.05$ ($p = 0.125$). Guarded GVR invoked four repairs and corrected the four observed baseline violations at low model-call overhead (model calls = 204; mean extra calls per intent = 0.02), yielding a paired risk difference estimate of 0.02 and an exploratory paired bootstrap CI = [0.005, 0.04] (bootstrap_seed = 1729, resamples = 10,000). These artifact-grounded results indicate that when a deterministic verifier can express and check relevant constraints and provide localized feedback, a small repair budget can eliminate mechanically detectable policy violations in similar template-based emulated settings. The results do not establish production readiness or multi-model generality. All reported numbers, provider accounting, validator traces, and analysis artifacts are drawn verbatim from the archived execution and analysis bundles referenced in the preregistration and execution manifests.

References [1] R. Mondal, A. Tang, R. Beckett, T. Millstein, and G. Varghese, "What do LLMs need to Synthesize Correct Router Configurations?", *Proceedings of the ACM*, 2023, doi: 10.1145/3626111.3628194. [2] R. Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments", *SSRN*, 2026, doi: 10.2139/ssrn.6754899. [3] Z. Ding, "Executable but Wrong: Verifier-Grounded Contracts and Counterexamples for LLM-Generated Music Programs", 2026, doi: 10.31224/7995. [4] C. Wang, M. Scazzariello, A. Farshin, S. Ferlin, D. Kostić, and M. Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?", *Proceedings of the ACM*, 2024, doi: 10.1145/3656296. [5] S. Y. Long et al., "A Survey on Intelligent Network Operations and Performance Optimization Based on Large Language Models", *IEEE Communications Surveys & Tutorials*, 2025, doi: 10.1109/comst.2025.3526606. [6] L. Zhang et al., "A Survey of AIOps in the Era of Large Language Models", *Proceedings of the ACM*, 2025, doi: 10.1145/3746635.

DISCLOSURE STATEMENT

Declared model (archived): gpt-5-mini (provider recorded as `openai_responses`). Adapter family: `hosted_netops_gvr_v1`. Deterministic validator: `deterministic_netops_validator_v5`. Task generator: `deterministic_netops_task_generator_v5`. Preregistration: C1-GVR-multivendor-2026-v1 (SHA-256

= 9e0fc992665835975b01289f7841926ef6abf3584252c5d0459ba0ecf166bb50), recorded prior to any model calls. Initial master prompt archived at `provenance/master_prompt.txt` (SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Human role: a human Research Director selected the topic family and constrained the initial master prompt prior to the autonomy lock; after the lock the AI autonomously performed literature verification, preregistration, experiment execution, analysis, peer review, revision, and manuscript drafting; no human manually edited technical manuscript content after the autonomy lock. Sampling semantics: execution used `shared_initial_candidate` (exactly one sampled initial generation per intent was produced and reused by both baseline and guarded); archived model configuration records `temperature = null/unset` (`null/unset` is not evidence of deterministic provider sampling and does not imply `temperature = 0`). Provider accounting (audited): `hosted_model_call_event_count = 204` decomposed as 200 shared initial generations + 4 repair calls. Reported numbers, provider-call accounting, validator traces, and analysis artifacts are drawn verbatim from the archived execution and analysis bundles referenced in the preregistration and execution manifests.

REFERENCES

- [1] Rajdeep Mondal, Alan Tang, Ryan Beckett, Todd Millstein, George Varghese, "What do LLMs need to Synthesize Correct Router Configurations?", 2023, 10.1145/3626111.3628194
- [2] Rafael Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments", 2026, 10.2139/ssrn.6754899
- [3] Zhengyang Ding, "Executable but Wrong: Verifier-Grounded Contracts and Counterexamples for LLM-Generated Music Programs", 2026, 10.31224/7995
- [4] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, Marco Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?", 2024, 10.1145/3656296
- [5] S.Y. Long, Jingjing Tan, Bomin Mao, Fengxiao Tang, Yangfan Li, Ming Zhao, Nei Kato, "A Survey on Intelligent Network Operations and Performance Optimization Based on Large Language Models", 2025, 10.1109/comst.2025.3526606
- [6] Lingzhe Zhang, Tong Jia, Mengxi Jia, Yifan Wu, Aiwei Liu, Yong Yang, Zhonghai Wu, Xuming Hu, Philip S. Yu, Ying Li, "A Survey of AIOps in the Era of Large Language Models", 2025, 10.1145/3746635